本地多模态助手 · 源码导读（新手版）

> 这份导读假设你**只学过 Python 基础语法**（会写函数、会 for 循环），
> 没写过后端、没碰过前端、没听过"流式"。
> 每一段我都按"**这是什么 → 逐行在干什么 → 它和谁有关**"来讲。
>
> 版本：2026-11-10（`d85587e` 之后）。行号会随改代码漂移，**函数名不会**，找不到时按函数名搜。

第一部分 · 先补 5 个概念（10 分钟）

不补这 5 个，后面每一行你都会卡住。

1. 前端 / 后端

- **后端**（`backend/` 里的 Python）：真正干活的，管模型、管文件、管工具。
- **前端**（`frontend/` 里的 HTML/CSS/JS）：只负责"画界面"和"把你说的话送过去"。

它们通过 **HTTP 接口**说话，就像点外卖：你在 App 里点单（前端发请求），厨房做菜（后端处理），把菜送回来（响应）。

本项目里，后端跑在 `127.0.0.1:8000`，前端就是浏览器里那个页面。

2. JSON —— 两边都能读的"纸条格式"

```
{"role": "user", "content": "你好"}
```

就是"键: 值"的文本。Python 里是 `dict`，JS 里是对象，**互换时都转成这段文本**。

后端往前端发消息、前端往后端发消息，全靠它。

3. 流式（streaming）—— 一边生成一边送

普通请求：后端想完 1000 字，**一次性**发给你 → 你要干等 20 秒，屏幕一片空白。

流式：后端每想出 3~5 个字就先发出来 → 你能看到字一个个"长"出来。

外卖类比：不是等整桌菜做好才上，而是**做好一道上一道**。

4. NDJSON —— 流式里怎么分条

流里没有"这是第一条、这是第二条"的边界。本项目的办法很土但很稳：

```
一行一个 JSON，行尾用 \n 隔开
{"message":{"content":"你"}}
{"message":{"content":"好"}}
{"__end":true,"answer":"你好"}
```

前端就按 `\n` 切开，一条条解析。**这就是为什么代码里到处是 `+ "\n"`** —— 那个换行是分隔符，不是排版。

5. 生成器 `yield` —— 后端的"挤牙膏"

```
def gen():
    yield "第一段"
    yield "第二段"
```

普通函数 `return` 一次就结束；`yield` 是**挤一下停一下**，调用方拿一段、再要下一段。

后端的流式就是靠它：每想出一片字，`yield` 一次，FastAPI 就立刻把它写进网络。

> ⚠️ 新手最容易踩的坑：`yield` 只能写在函数里，而且一旦写了 `yield`，这个函数就**变成生成器**，不能再 `return 具体值`（本项目为此挂过一次服务器，见第三节末尾）。

第二部分·程序是怎么起来的（`run.py`，327 行）

打开 `run.py`，从上往下看。

第 1 段：开头那个 `if sys.stdout is None`

```
if sys.stdout is None or sys.stderr is None:
    _f = open(os.path.join(_log_dir, "app.log"), "a", encoding="utf-8")
    if sys.stdout is None:
        sys.stdout = _f
```

- `sys.stdout` 就是 `print()` 往哪写。有控制台时它是一块屏幕；**没有控制台时它是 `None`**。
- 双击桌面快捷方式用的是 `pythonw.exe`（不带黑窗口），此时 `sys.stdout` 就是 `None`。
- 于是任何一句 `print("...")` 都会变成 `None.write(...)` → **直接抛异常 → 进程秒退**。
- 表现出来就是：**双击图标，窗口闪一下没了，界面也发不出消息。**

它和谁有关： 后端代码里有大量 `print()` 和日志（`backend/main.py`、`tools.py` 到处是）。所以这个兜底必须放在**最前面**，早于任何 import 后的执行。

> 新手理解：这不是"优雅设计"，是**被坑出来的**。注释里写的日期，就是被坑的那天。

第 2 段：`main()` —— 三件事（242 行起）

```
server = uvicorn.Server(uvicorn.Config(app, host=HOST, port=PORT, log_level="info"))
t = threading.Thread(target=_serve, args=(server,), daemon=True)
t.start()
```

逐行：

行	在干什么	为什么
262	创建 uvicorn 服务对象	uvicorn 是"把 Python 函数变成网站"的工具
264	塞进一个**后台线程**	因为主线程要留给窗口，不然后面 `webview.start()` 会把服务堵死
265	启动线程	此刻服务开始监听 8000
267	`_wait_until_up()` 轮询端口	起服务是异步的，**必须等它真的能连上**再开窗口，否则窗口里是空白
297-300	导入 `webview` 开窗口	pywebview 用系统自带浏览器内核做个独立窗口

它和谁有关：

- `app` 是从 `backend.main` 导入的 FastAPI 对象 —— 也就是**整个后端的总入口**。
- 端口 8000 定死在 `HOST/PORT` 常量里，前端 `app.js` 里的请求都是相对路径（`/api/chat`），所以它们天然对得上。

新手常见误解： 以为"开了窗口就是在跑前端"。其实窗口只是个浏览器，**真正干活的是那个后台线程**。窗口关掉 → `webview.start()` 返回 → 主线程结束 → 进程退出 → 服务跟着没了。

第三部分·界面长什么样（`frontend/index.html`，469 行）

结构（只看 `<body>` 就够）

```
<aside class="sidebar">          左侧栏
    ├ 会话列表
    ├ 知识库面板
    ├ 生成文库面板          ← 模型写出来的文件都在这
    ├ 图片库
    └ 定时任务
<main class="main">              右侧主区
    ├ <header> 顶栏：☰ / 模型下拉 / 🔑 Key / 主题
    ├ 消息区 #messages
    └ 输入区：文本框 + 发送
```

关键元素（后面代码会一直提到）

id	是什么
`#modelSelect`	模型下拉。**6 个选项 = 3 个本地 + 3 个云端**，值分别是 `auto` / `qwen3-vl:30b` / `qwen3-vl:8b` / `deepseek-flash` / `deepseek-flash-think` / `deepseek-v4-pro`
`#sendBtn`	发送按钮，绑定 `send()`
`#input`	输入框
`#messages`	消息区，所有气泡都往里 append
`#uiVer`	标题旁的小版本号（用来确认窗口跑的是哪份 JS）

> 你在 11-09 那版里看到过"顶栏下拉停在本·自动，但发不出去"——根因就在这个下拉的 `auto` 值上，见第八节。

脚本是怎么加载的

```
<script>
  var v = String(Date.now());
  document.write('<script src="/static/app.js?v=' + v + '"><\/script>');
</script>
```

为什么加 `?v=时间戳`： WebView2 会把 `app.js` 缓存到磁盘，后端明明发了新文件，窗口还在跑旧的。加个时间戳，每次 URL 都不同，缓存物理上命不中。

第四部分·点"发送"之后（`frontend/app.js`，6430 行）

这是全项目最该读懂的一段。**从 3093 行 `async function doSend` 往下一路读。**

4.1 第一道门：`send()`（约 3060 行）

```
if (!text && imgB64.length === 0 && !videoB64) return;
if (streaming) {
  const idle = Date.now() - (window.__lastStreamAt || 0);
  const stuck = !abortCtl || idle > 60000;
  if (stuck) { /* 复位后继续发 */ } else { showToast("上一条还在生成…"); return; }
}
```

逐行：

1. 什么内容都没有 → 直接返回（防止发空消息）。
2. 上一轮还在跑 → 判断"是真在跑还是卡死了"：超过 60 秒没有任何数据、或者连中断句柄都没了 → 认为卡死，**复位后继续发送**。

为什么要有这段： 老代码是 `if (... || streaming) return;` —— 一旦 `streaming` 卡住（上一轮异常没复位），**点发送等于什么都没发生**，也不报错。这就是"桌面端发不出去、手机端正常"的真相之一。

4.2 `doSend()` 开头：建两个气泡

```
const thinkWrap = document.createElement("div");   // 思考过程气泡
const answerBubble = document.createElement("div"); // 回答气泡
messagesEl.appendChild(thinkWrap);
```

- 思考气泡是折叠的（点标题展开），因为推理模型会吐几千字"心理活动"。
- 回答气泡才是你看到的那块。

4.3 选哪条通道（约 3925 行）

```
let _prov = _msEl ? _msEl.value : "auto";
if (_prov === "auto" && /^deepseek/i.test(window.__defaultModel || "")) {
  _prov = String(window.__defaultModel);
}
const _isDS = _prov.indexOf("deepseek") === 0;
const _chatUrl = _isDS ? "/api/chat/deepseek" : "/api/chat";
```

这段就是"云端还是本地"的岔路口：

- 下拉选的是云端 → 打 `/api/chat/deepseek`（带 Key）。
- 选的是本地 → 打 `/api/chat`（走 Ollama）。

- 选的是「本地·自动」→ 看 `window.__defaultModel`（启动时从 `/api/config` 读的默认模型）。**如果默认模型是 deepseek-*，就自动改走云端** —— 否则会出现"拿云端模型名去问本地 Ollama"的怪事（真发生过）。

4.4 发请求 + 读流（约 3985 行起）

```
buf += decoder.decode(value, { stream: true });
const lines = buf.split("\n"); buf = lines.pop();
for (const line of lines) {
  let obj; try { obj = JSON.parse(line); } catch { continue; }
  if (obj.message?.content) { answer += obj.message.content; paintAnswer(false); }
}
```

这是流式的核心，逐行解释：

行	在干什么	为什么要这么写
`decoder.decode(..., {stream:true})`	把字节流解成字符串	中文一个字 3 字节，**很可能被切在两片之间**；`stream:true` 让解码器记住"半个字"，下次拼上
`buf.split("\n")`	按换行切成若干条	NDJSON：一行一条
`lines.pop()`	最后一条很可能只有半行	**留在 `buf` 里等下一片**，不能当场解析
`JSON.parse` 包在 try 里	解不出来就跳过	网络抖一下不该让整轮崩掉
`answer += content`	把碎片拼成完整正文	这个变量最后要交给渲染
`paintAnswer(false)`	画到屏幕上	传 `false` 表示"可以节流"（120ms 一次），否则每个字都重排 DOM 会卡

新手常见误解： 以为"流式 = 后端推一次，前端接一次"。实际是**上千次**，每次几个字。所以每一行都必须能容忍"断在半路"。

4.5 结束信号 `__end`（约 4010 行）

```
if (obj.__end) {
  if (typeof obj.answer === "string" && obj.answer) {
    answer = obj.answer;      // 用服务端的整段正文覆盖
    paintAnswer(true);        // 立刻重绘一次
  }
  continue;
}
```

为什么要有这个： 前面的 `answer` 是"一片片拼出来的"，只要漏一片就少字。所以后端在结束时**把整段正文再发一遍**（`__end.answer`），前端直接用它覆盖 —— 相当于交卷前对答案。

它和谁有关： 后端 `main.py` 的云端通道 `gen()` 末尾（约 6720 行）就是这个 `__end` 的生产者。

4.6 收尾渲染 `renderFinal()`（1196 行）

```
function renderFinal(bubble, text, userText) {
  const raw = String(text == null ? "" : text);
  if (isPlainMode()) { bubble.textContent = raw; return; }
  try {
    if (raw.indexOf("```") >= 0) renderAnswerWithCode(bubble, raw, userText || "");
    else renderAnswerLinks(bubble, raw);
  } catch (e) { bubble.textContent = raw; return; }

  const wantTail = raw.replace(/\*\*|`|^\s*[-*]\s+/gm, "").trim().slice(-24).replace(/\s+/g, "");
  const shown = (bubble.innerText || "").replace(/\s+/g, "");
  if (wantTail && shown && !shown.includes(wantTail)) {
    bubble.textContent = raw;          // 渲染丢了结尾 → 退回纯文本
  }
}
```

逐段：

1. **纯文本模式**：直接 `textContent`（屏幕上就是原文，不可能少字）。
2. **格式模式**：`mdToHtml()` 把 `**粗体**`、`- 列表` 变成 HTML。
3. **末尾 24 字核对**：把原文最后 24 个字（去掉 markdown 符号）拿去屏幕上找，**找不到就说明渲染吃了尾巴** → 直接退回纯文本。

它和谁有关：

- `isPlainMode()`（1182 行）决定走哪条路，默认格式模式，地址栏 `?plain=1` 可切纯文本。
- `addMsg()`（1222 行）**历史消息也走这个函数** —— 早期只有实时消息走，导致"第一句完整、第二三句断"，这是真 bug。

4.7 最后一道保险：回读服务端

```
const d = await api("/api/sessions/" + sessionId);
const last = [...(d.messages || [])].reverse().find(m => m.role === "assistant");
if (String(last.content).length > String(answer).length) {
  answer = last.content;
  renderFinal(answerBubble, answer, promptText);   // 用服务端的重绘
}
```

为什么： 服务端自己存了一份完整原文（见第六节）。只要它比屏幕上的长，就以它为准。

第五部分·后端收到请求（`backend/main.py`，9133 行）

两个端点，长得不一样但套路相同。

5.1 本地通道 `@app.post("/api/chat")`（4308 行）

```
model = req.model or cfg["default_model"]
if str(model).strip().lower() in ("", "auto"):
    model = cfg["default_model"]

if str(model).strip().lower().startswith("deepseek"):
    def _ds_mismatch():
        yield json.dumps({"error": "当前「本地·自动」跟随的配置是云端模型…", "__end": True}) + "\n"
    return StreamingResponse(_ds_mismatch(), media_type="application/x-ndjson")
```

这里有个必须知道的坑：

- 这个函数叫 `async def chat(...)`，它正常是 `return StreamingResponse(...)`（返回一个"响应对象"）。
- 但如果你在里面写 `yield`，Python 会把它**整体变成生成器**，末尾的 `return StreamingResponse(...)` 就变成语法错误：`'return' with value in async generator`。
- 后果：**整个后端起不来**（nginx 直接 502）。我昨晚就是这么把服务器搞挂 25 分钟的。

所以上面这段用的是"**定义一个小生成器，然后 return 它**"，不是直接 yield。你以后要在这个函数里加流式输出，必须照这个写法。

5.2 云端通道 `@app.post("/api/chat/deepseek")`（6552 行）

结构：

```
async def gen():
    ...
    while True:
        kind, val = await loop.run_in_executor(None, _qget)   # 从队列取一条
        if kind == "content":
            _ACC.append(val)                                    # 攒起来
            yield json.dumps({"message": {"content": val}}) + "\n"   # 推给前端
    # 结束前：补句、落盘、发 __end
```

几个设计要点，逐个说：

设计	为什么
用 `queue.Queue` + 线程	真正干活的 `deepseek_client` 是同步阻塞的（等网络），塞进线程池才不会堵住事件循环
`_qget` 里 `q.get(timeout=15)`	15 秒没数据就发一个 `{"ping":1}` 心跳，否则前端会以为卡死
`_ACC` 攒全文	结束时要用它拼 `__end.answer`
结束时 `sessions.save_messages(...)`	**服务端自己存一份**，不等前端上传

5.3 `__end` 与"五层不丢字"

```
_full = "".join(_ACC)
# 补句：结尾没有句号就再问一次"把话说完"，最多 3 轮
...
yield json.dumps({"__end": True, "answer": _full}, ensure_ascii=False) + "\n"
```

五层叠起来是这样（从上到下越来越靠后）：

层	在哪	作用
1	`deepseek_client` 的 `content_buf`	生成侧攒全，不受网络影响
2	这里的 `__end.answer`	整段再发一遍，前端不靠拼
3	这里的 `sessions.save_messages`	磁盘上永久一份
4	`session_save()`（8786 行）"只许增长"	前端上传短版本时，不让它覆盖长的
5	`renderFinal()` 尾部核对 + 回读	屏幕与原文逐字比对

第六部分·云端通道内部（`backend/deepseek_client.py`，746 行）

从 355 行 `chat_stream_with_tools` 开始读，这是最值得逐行看的地方。

6.1 一轮请求的骨架

```
for rnd in range(max_rounds):
    body = {"model": wire_model, "messages": msgs, "stream": True, ...}
    with requests.post(url, json=body, stream=True) as resp:
        for raw in resp.iter_lines():
            line = raw.decode().strip()
            if not line.startswith("data:"): continue
            obj = json.loads(line[5:])
            delta = obj["choices"][0]["delta"]
            if delta.get("content"):
                content_buf.append(delta["content"])
                yield ("content", delta["content"])
```

逐行：

- `max_rounds`：一个提问可能要调好几次工具，所以外面套一层"轮次"循环。
- `iter_lines()`：DeepSeek 返回的是 SSE（`data: {...}` 一行一行的），所以要 `line[5:]` 去掉 `data:` 前缀。
- `yield ("content", ...)`：**元组**，第一个元素是"类型标签"，第二个是内容。上层 `main.py` 靠这个标签决定推给前端哪个字段。

6.2 工具调用是分片的（最容易看错的地方）

```
for tc in (delta.get("tool_calls") or []):
    idx = int(tc.get("index") or 0)
    slot = tool_calls.setdefault(idx, {"id": "", "name": "", "args": ""})
    if tc.get("id"):         slot["id"] = tc["id"]
    fn = tc.get("function") or {}
    if fn.get("name"):       slot["name"] += fn["name"]
    if fn.get("arguments"):  slot["args"] += fn["arguments"]
```

为什么用 `+=`： 模型调工具时，参数字符串是被切成几十片的，比如：

```
片1: {"pa
片2: th": "con
片3: fig.json"}
```

必须按 `index` 归组、**字符串累加**，最后才是完整 JSON。

6.3 结束时两种特殊处理

```
if finish == "length":
    # 被长度上限截断 → 自动续写（把已写的当历史，让它接着写）
```

```
if guard is not None and not guard_used and content_buf:
    tip = guard(used, "".join(content_buf))
    if tip:  # 模型"光嘴上说说"（说要写文件却没调工具）→ 退回一轮
```

它和谁有关： `guard` 是 `main.py` 传进来的函数 `_ds_guard_factory(last_user)`，它判断"用户要文件，模型却只写了正文"。

第七部分・工具是怎么被真正调用的

模型说"我要读文件"≠ 文件被读。中间有一段硬逻辑，在 `backend/main.py` 的工具循环里（`chat_stream_with_tools` 返回 `tool_calls` 之后）。

```
模型输出 tool_calls（JSON）
   ↓ main.py 解析出 name + arguments
   ↓ 查表：name → 对应 tools.py 里的函数
   ↓ 调用（可能要 30 秒，比如跑代码）
   ↓ 把结果作为 role="tool" 的消息塞回对话
   ↓ 再请求模型一次，让它根据结果继续写
```

举例：用户说"读一下 config.json"

1. 模型返回 `{"name":"read_file","arguments":"{\"path\":\"config.json\"}"}`
2. `main.py` 调到 `tools.py` 的 `_do_read_file`
3. `tools.py` 里做**越界检查**（不许读项目根目录以外的文件）
4. 返回文件内容（可能几百行）
5. 这段内容作为工具结果回灌给模型
6. 模型看到内容后，才写出"config.json 里 default_model 是 …"

安全边界在 `tools.py`，不在模型。 模型说"读 /etc/passwd"，代码层会拒绝 —— 这是本项目最重要的一条设计。

第八部分 · 三个"看起来像 bug"的地方（新手必看）

8.1 下拉停在「本地•自动」，却发不出去

- `auto` 的意思不是"随便挑一个"，而是"**跟随配置里的默认模型**"。
- 昨晚我把默认模型改成了 `deepseek-flash`（云端），但 `auto` 仍然被送到 `/api/chat`（本地 Ollama）→ 拿云端模型名问本地 → 卡在"正在加载模型"。
- 现在前后端都补了判断：`auto` 指向云端模型时，自动改走 `/api/chat/deepseek`；后端也会立刻返回可读错误，不再傻等。

8.2 回答"最后几个字没了"

按可能性排序，**先看日志再动手**：

1. `app.log` 里 `[ds] 本轮 finish=stop 正文=N 字 尾='…'` → 后端写了多少、结尾是什么
2. Console 里 `[stream] __end local=… server=…` → 前端收到多少
3. 两个数相等但屏幕短 → **渲染层**（`renderFinal` 会自己核对并退回纯文本）
4. `local < server` → **传输层**（被掐断/超时）

> 这条排查顺序比"猜"快 10 倍，务必记住。

8.3 "改了代码但界面没变"

三个可能，按顺序查：

1. 窗口没重载 → 顶栏看 `v……` 版本号是否变了
2. WebView 缓存 → 现在脚本带 `?v=时间戳`，理论上不会再中
3. 改的是**后端** → 后端必须重启进程，热更无效

第九部分 · 动手实验（比读十遍有用）

按顺序做，每个 5 分钟：

实验 1 | 看流式到底有多碎

在 `app.js` 的 `answer += obj.message.content;` 上面加一行：

```
console.log("[片]", JSON.stringify(obj.message.content));
```

发一句话，打开 F12 看控制台。你会看到几十上百条，每条一两个字 —— **这就是"流式"的真面目。**

实验 2 | 让后端多发一条状态

在 `main.py` 云端 `gen()` 里，`yield __end` 之前加：

```
yield json.dumps({"note": "本轮共 %d 字" % len(_full)}) + "\n"
```

重启后端，发消息，右上角会弹出这条提示。

实验 3 | 故意制造一次"少字"

把 `paintAnswer` 里的 `answerBubble.textContent = answer;` 改成 `answer.slice(0, -5)`（少最后 5 个字）。发消息 —— 你会发现最终显示其实没少，因为 `renderFinal` 的最后一道核对把它补回来了。**这一步能让你真正理解"五层兜底"为什么要叠。**

实验 4 | 读一个工具

打开 `backend/tools.py`，搜 `def _do_read_file`，从头读到尾（大概 40 行）。看它怎么校验路径、怎么处理编码、出错返回什么格式。

第十部分 · 术语表

词	一句话解释
uvicorn	把 Python 函数变成 HTTP 服务的工具
FastAPI	写接口的框架，`@app.post("/api/chat")` 就是在注册接口
StreamingResponse	FastAPI 的"流式响应"，配合 `yield` 用
NDJSON	一行一个 JSON，本项目前后端的分条格式
SSE	服务商（DeepSeek）返回流的格式，`data: {...}`
`yield`	挤牙膏式返回；写了它函数就变成生成器
`tool_calls`	模型请求调用工具的字段，**分片**传输
token	模型的计费/长度单位，中文约 1 字 ≈ 1~1.5 token
pywebview	用系统浏览器内核做桌面窗口的库
WebView2	Windows 自带的 Chromium 内核，本项目窗口就是它

附录 · 行号速查（按这个顺序读）

```
run.py:19              pythonw 无 stdout 的兜底（先看这段，理解"为什么要防御"）
run.py:242             main(): 起服务 → 等端口 → 开窗口
index.html:318         #modelSelect 的 6 个选项
index.html:456         脚本加载（带 ?v= 时间戳）
app.js:1182            isPlainMode()
app.js:1196            renderFinal()   ← 渲染 + 尾部核对
app.js:1222            addMsg()        ← 历史消息也走同一套
app.js:3060            send() 的卡死自愈
app.js:3093            doSend()        ← 主线从这里开始
app.js:3925            云端/本地分流
app.js:3985            读流 + 按行解析
app.js:4010            __end 覆盖正文
main.py:4308           /api/chat（本地）
main.py:4775           本地通道 gen()
main.py:6552           /api/chat/deepseek（云端）
main.py:6671           云端通道 gen()   ← 心跳、补句、落盘、__end
main.py:8786           PUT /api/sessions/{sid}  ← 只增不减
deepseek_client.py:355   chat_stream_with_tools()   ← 最值得逐行
tools.py               _do_ 开头的函数，挑 5 个看
selfdev.py             自我迭代：备份 → 改 → 语法校验 → 失败回滚
keyvault.py:1          94 行，全读
scheduler.py:428       run_now(): 后台执行 + 立即返回
```

附：三天节奏

时间	做什么
Day 1 上午	第一部分 5 个概念 + 第二部分 run.py（读懂"防御式编程"）
Day 1 下午	第三、四部分：`index.html` 结构 + `app.js` 的 `doSend` 主线
Day 1 晚上	第五部分：两个端点，对照 `__end` 看前端怎么接
Day 2 上午	第六部分：`chat_stream_with_tools` 逐行（这段最值钱）
Day 2 下午	第七部分 + `tools.py` 挑 5 个 `_do_` 函数
Day 3 上午	第八部分三个坑 + 第九部分四个实验
Day 3 下午	自己加一个小功能（比如给某个工具多加一行日志），跑通

读完不等于懂了，改通才算。 建议第一个改动从"实验 2"开始 —— 它只加一行，但会让你跑通"改后端 → 重启 → 看到效果"的完整链路。